Runbook — Confluent Cloud for Apache Flink (CCAF)

End-to-end operational guide for running the `confluent-kafka-isotope` reports on **Confluent Cloud for Apache Flink (CCAF)**: provision → deploy reports → drive traffic → observe → teardown. Unlike the Confluent Platform + Flink on Minikube, this is **Terraform-driven** — no local cluster — and everything runs in a fresh Confluent Cloud environment under terraform/.

> This is the **managed CCAF** path. The self-managed **Confluent Platform + Flink on Minikube** path is in docs/runbook-minikube.md. Both runtimes run the same seven reports from the same PTF JAR — see root README §3.4 for the format-by-runtime split.

Table of Contents

1.0 Prerequisites

- Terraform `>= 1.13` installed locally.
- A **Cloud-level** Confluent Cloud API key (not cluster-scoped) with permission to create environments, Kafka clusters, Flink compute pools, service accounts, role bindings, Flink artifacts, and statements. Generate via Console → Settings → Cloud API keys.

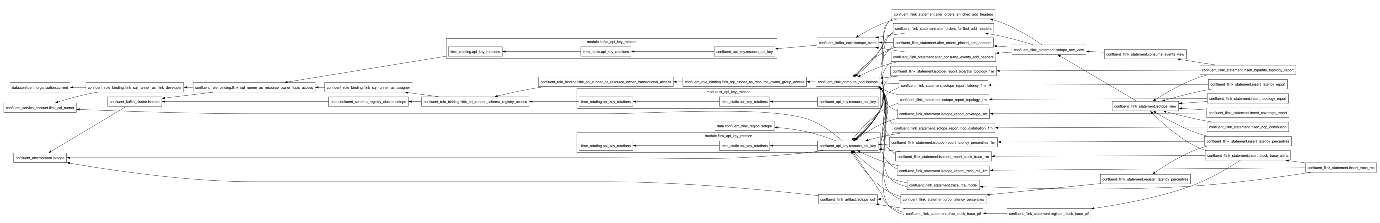
2.0 Provision + deploy the reports

```
make cc-flink-reports-up CONFLUENT_API_KEY=$CONFLUENT_API_KEY
CONFLUENT_API_SECRET=$CONFLUENT_API_SECRET
```

This runs scripts/deploy-cc-flink-reports.sh `create`, which builds the PTF shadow JAR if missing, then runs `terraform apply -auto-approve` in terraform/.

- **First run takes ~6–8 minutes** (Kafka cluster provisioning dominates).
- **Re-applies are idempotent** — `CREATE … IF NOT EXISTS` plus `lifecycle { ignore_changes = [compute_pool] }` on every statement.
- Optional retention override: the script accepts `--day-count=<DAYS>` (default `30`); both `make` targets require `CONFLUENT_API_KEY` / `CONFLUENT_API_SECRET` or they fail fast with a clear message.

3.0 What gets created



Resource	Name	Notes
`confluent_environment`	`confluent-kafka-isotope`	ESSENTIALS stream-governance package
`confluent_kafka_cluster`	`kafka-isotope`	Standard, single-zone, AWS us-east-1 by default
`confluent_kafka_topic` × 4	`orders.{placed,enriched,fulfilled}` + `isotope_consume_edge_markers`	Only the event + consume-marker topics are pre-created. The 7 `isotope_report_*_1m` sink topics are created on first deploy by their `CREATE TABLE` (pre-creating them would make CCAF's Topic Catalog auto-import them as `(key BYTES, val BYTES)` and silently no-op the typed DDL).
`confluent_service_account` + 6 role bindings	`isotope-flink-sql-runner`	FlinkDeveloper (org) + ResourceOwner on topic/transactional-id/group/SR-subject `*` + Assigner
`confluent_flink_compute_pool`	`isotope-flink-statement-runner`	10 CFU; headroom for 7 INSERTs + ad-hoc SELECTs
`confluent_flink_artifact`	`isotope-flink-udf`	Uploads `ptf/build/libs/isotope-flink-udf.jar`
`confluent_flink_statement` × 25	(see file)	4 ALTER TABLE + 3 VIEW + 7 sink CREATE TABLE + 2 CREATE FUNCTION (both PTFs) + 7 INSERT INTO (23 long-lived) + 2 transient DROP FUNCTION
`confluent_flink_statement` × 3 (optional)	(see terraform/setup-ccaf-ai.tf)	Optional AI trace-RCA report — `CREATE MODEL trace_rca` + 1 Protobuf sink + 1 `INSERT … ML_PREDICT`. **Gated on `var.enable_trace_rca` (default `false`)**, so a normal apply skips them entirely. Set `rca_model_api_key` (and `rca_model_provider`/`_version`/`_endpoint` for a non-OpenAI provider) to enable. See root README §3.4.

Two rotating service-account API key pairs (one Kafka, one Schema Registry) are managed by `module.kafka_api_key_rotation` and `module.sr_api_key_rotation` in terraform/setup-confluent-kafka.tf.

4.0 Useful outputs

```
terraform -chdir=terraform output environment_id
terraform -chdir=terraform output kafka_bootstrap_servers
terraform -chdir=terraform output schema_registry_url
terraform -chdir=terraform output -raw kafka_api_key      # sensitive
terraform -chdir=terraform output -raw kafka_api_secret   # sensitive
```

5.0 Drive traffic (required to see report rows)

The demo app runs **on your host** and talks to Confluent Cloud over SASL_SSL. You don't export credentials manually: scripts/cc-app-run.shsources scripts/cc-cli-env.sh, which pulls the Kafka + Schema-Registry credentials from `terraform output`, builds the JAAS string, and invokes `./gradlew :app:run` with the six `-D` flags. It hard-fails if any of the seven required values is missing.

Run the 4-stage demo, one verb per terminal (same A/B/C/D order as the local demo):

```
scripts/cc-app-run.sh place 'hello'    # A — kick the chain off (orders.placed)
scripts/cc-app-run.sh enrich           # B — orders.placed   → orders.enriched
scripts/cc-app-run.sh fulfill          # C — orders.enriched → orders.fulfilled
scripts/cc-app-run.sh ship             # D — terminal consume orders.fulfilled (emits
marker)
```

`cc-app-run.sh` also accepts the generic `send` / `hop` / `consume` / `sink` passthrough for ad-hoc topics — run it with no args for the full verb list.

Sustained traffic. The report jobs aggregate over `TUMBLE(event_time, INTERVAL '1' MINUTE)` windows, which only emit once the watermark advances past `window_end`. A burst inside a single 1-minute interval sits in one open window forever — spread records across **multiple** windows:

```
# 30 records, 5s apart ≈ 2.5 min of event-time → spans 3+ windows
for i in {1..30}; do scripts/cc-app-run.sh place "burst-$i"; sleep 5; done
```

Wait ~90s after the **last** record before checking results — that's the watermark catching up. `stuck_trace_alerts_1m` only fires for a trace that goes ≥60s of event time without a fresh hop; to exercise it, send one record to `orders.placed`, skip the `enrich`/`fulfill` hops, and keep sending unrelated records so the watermark crosses `event_time + 60s`.

6.0 Observe

In the Cloud Console **Flink SQL workspace**, query the report tables:

```
SELECT * FROM isotope_report_latency_1m;
SELECT * FROM isotope_report_bipartite_topology_1m;   -- all 6 edges: 3 produce + 3
consume
SELECT * FROM isotope_report_hop_distribution_1m;
-- …coverage, stuck_trace, topology, latency_percentiles
```

Terminal D prints the same `trace_id` across all three hops. CCAF report topics ride **Protobuf+SR** (`proto-registry`), versus Avro+SR on the CP path.

7.0 Teardown

```
make cc-flink-reports-down CONFLUENT_API_KEY=$CONFLUENT_API_KEY
CONFLUENT_API_SECRET=$CONFLUENT_API_SECRET
```

Runs `terraform destroy -auto-approve` — deletes **every** resource above, including the environment itself (the report sink topics are cleaned up via the environment cascade). Safe to run repeatedly.

> **Cost note:** the Kafka cluster and compute pool bill while they exist. Teardown when you're done rather than leaving the environment running.

8.0 Troubleshooting

- **`CONFLUENT_API_KEY … must be set`.** The `make` target requires both vars on the command line (or exported and passed through) — see [§1.0 Prerequisites](#).
- **`Aggregate functions are not supported`.** Expected if you try to add a UDAF — CCAF rejects user-defined aggregates, which is why percentiles is a PTF — see [root README §3.4](#).
- **Typed `CREATE TABLE` silently no-ops.** A report sink topic was pre-created, so the Topic Catalog auto-imported it as `(key BYTES, val BYTES)` — don't pre-create the `isotope_report_*_1m` topics — see [§3.0 What gets created](#).

- **No report rows.** Almost always the watermark — traffic must span multiple 1-minute windows; wait ~90s after the last record — see [§5.0 Drive traffic].
- **App can't authenticate.** `cc-cli-env.sh` couldn't read `terraform output` — re-run `make cc-flink-reports-up` so the rotated keys exist, then retry.